

工具方案

项目名称： 数据解析软件自动分析
及模版生成工具

研制单位： 北京理工大学

负 责 人： 黄永刚

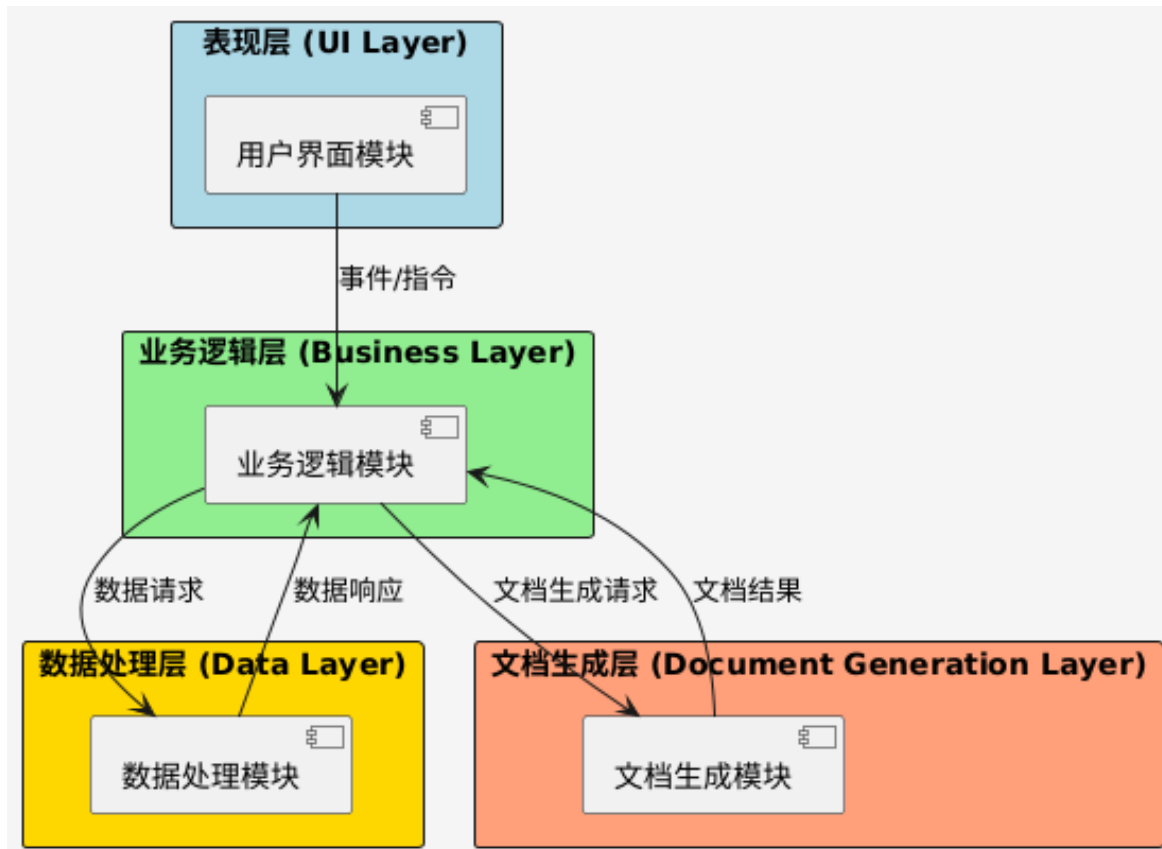
日 期： 2025 年 6 月 12 日

1 设计方案

- 1.1 系统框架
 - 1.1.1 核心模块构成
 - 1.1.2 关键技术特性
- 1.2 技术框架
 - 1.2.1 技术栈
 - 1.2.2 关键技术实现
- 1.3 功能设计
 - 1.3.1 文档生成功能
 - 1.3.1.1 设计思想
 - 1.3.1.2 详细设计
 - 1.3.2 需求管理功能
 - 1.3.2.1 设计思想
 - 1.3.2.2 详细设计
 - 1.3.3 函数关联功能
 - 1.3.3.1 设计思想
 - 1.3.3.2 详细设计
- 1.4 前端设计
 - 1.4.1 界面布局设计
 - 1.4.1.1 设计思想
 - 1.4.1.2 详细设计
 - 1.4.2 交互控件设计
 - 1.4.2.1 设计思想
 - 1.4.2.2 详细设计

1 设计方案

1.1 系统框架



系统整体框架如图所示，采用分层架构设计范式，其核心思想源于软件工程中“关注点分离”（SOC）原则，通过层次化的功能解耦提升系统的可维护性、可扩展性与可测试性。系统明确划分为：

- 表现层（UI Layer）
- 业务逻辑层（Business Layer）
- 数据处理层（Data Layer）
- 文档生成层（Document Generation Layer）

其中，表现层聚焦用户交互的可视化呈现，业务逻辑层承载领域规则的抽象与执行，数据处理层负责底层数据的全生命周期管理，文档生成层则专注于结构化输出的格式控制与内容合成。各层通过定义清晰的接口契约实现松耦合通信，例如表现层通过事件驱动机制向业务逻辑层传递用户操作指令，业务逻辑层通过数据传输对象（DTO）与数据处理层进行数据交换，文档生成层通过模板引擎接口获取业务逻辑层提供的元数据。这种分层设计不仅符合企业级应用的典型架构模式，更通过层次间的职责隔离降低了系统的整体复杂度。

1.1.1 核心模块构成

- 用户界面模块

采用JavaFX框架实现图形化界面

实现了树形结构展示、表格操作、文件浏览等功能组件

支持实时交互和动态更新

采用响应式设计,确保界面布局灵活可调

- 数据处理模块

实现了多叉树形数据结构(TreeNode)的构建与维护

提供数据去重、过滤、转换等核心功能

支持数据的序列化与反序列化

- 文档生成模块

基于Apache POI实现Word文档操作

支持模板化文档生成

实现了多线程并行处理以提升性能

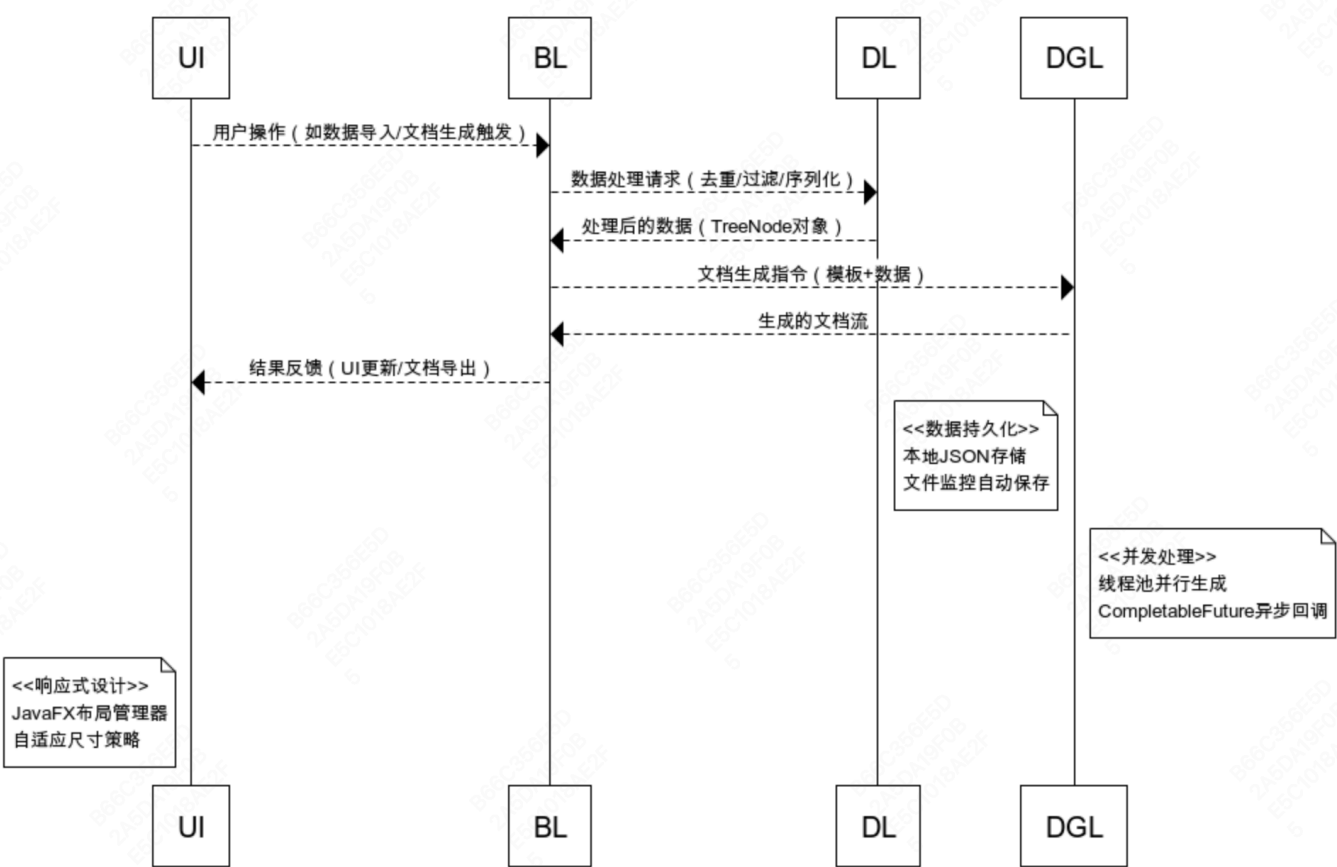
- 业务逻辑模块

实现需求管理与追踪

提供模块划分与关联关系管理

支持详细设计文档自动生成

实现了变更管理与版本控制



1.1.2 关键技术特性

并发处理：采用JDK的ExecutorService框架实现线程池管理，通过配置不同线程池适应不同负载场景（如文档生成的计算密集型任务使用固定池，短期异步任务使用缓存池）。异步操作的进度监控通过Future接口与Callable任务实现，结合JavaFX的Platform.runLater()方法将进度更新同步至UI线程；线程安全的数据访问则通过ReentrantLock与ConcurrentHashMap等并发工具类保障，避免多线程环境下的数据竞争。

事件处理：基于观察者模式构建事件响应机制，通过自定义事件类（如DataChangeEvent、DocumentGenerateEvent）与事件总线（Event Bus）实现组件间解耦通信。UI事件（如按钮点击）通过JavaFX的EventHandler接口捕获并封装为事件对象，由事件总线分发至订阅组件（如数据处理模块或文档生成模块）；异步事件处理则结合CompletableFuture实现非阻塞执行，例如耗时的数据过滤操作被提交至后台线程执行，完

成后通过事件回调更新 UI 状态。

错误处理：设计了统一的异常处理框架，通过自定义业务异常类（如 `DataValidationException`、`DocumentGenerateException`）封装不同场景的错误类型，并结合全局异常处理器（`UncaughtExceptionHandler`）捕获未处理异常。错误日志记录采用 SLF4J 日志门面与 Logback 实现，支持日志级别控制（DEBUG/INFO/ERROR）与日志文件滚动策略；用户友好提示则通过 JavaFX 的 Alert 对话框与状态提示条实现，确保错误信息的可读性与操作指引的明确性。

数据持久化：支持本地文件存储（默认存储路径为用户目录下的 `.projects` 文件夹），采用 JSON 格式（结合 Jackson 的 `ObjectMapper`）存储项目元数据与树形结构数据；数据备份与恢复机制采用快照策略，通过定时任务生成历史版本，用户可通过版本管理界面选择任意快照进行恢复。

1.2 技术框架

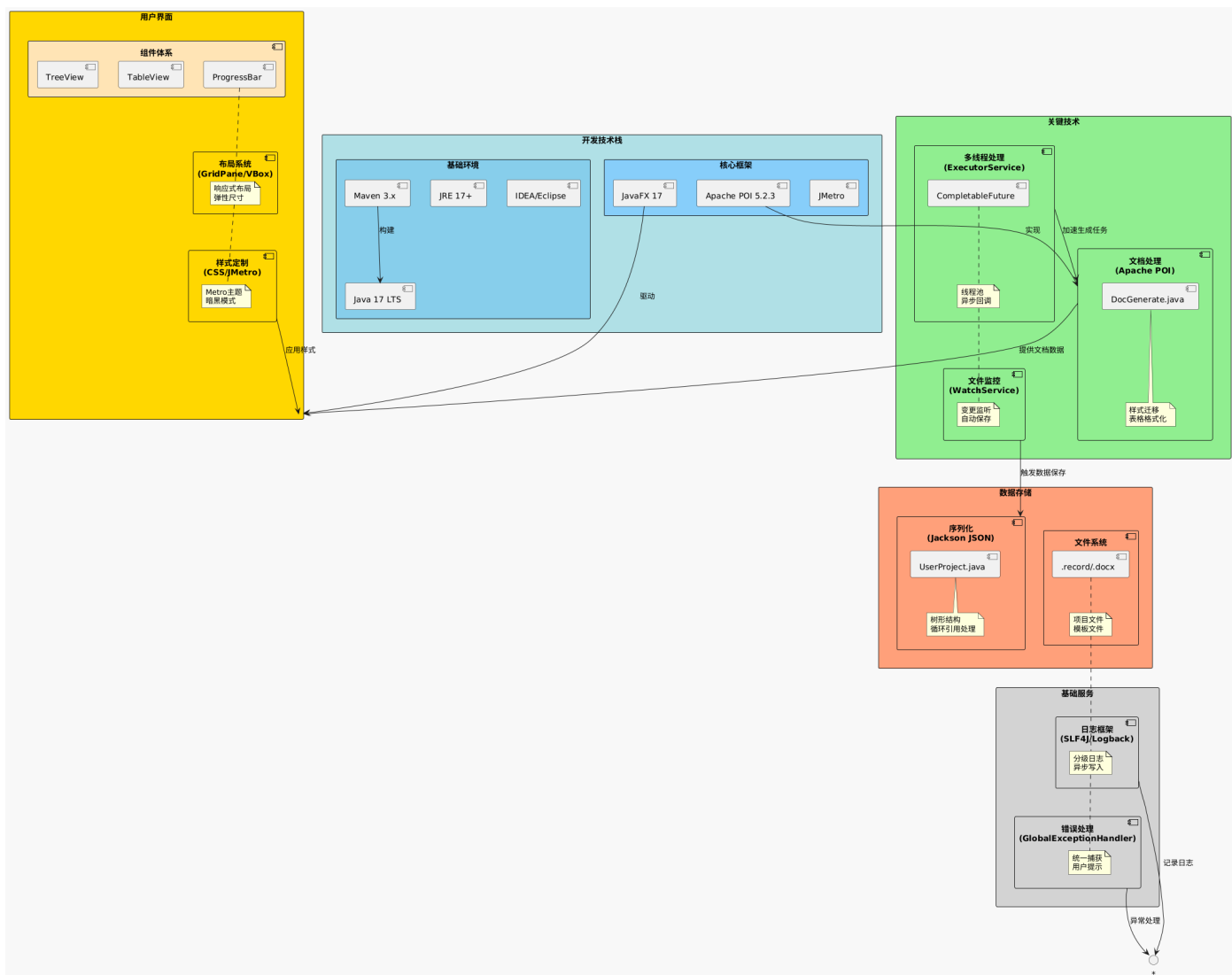
1.2.1 技术栈

系统基于 Java 生态构建现代化技术栈，遵循企业级应用开发的标准化规范，融合模块化开发与跨平台部署能力。

具体的，系统采用 **Java 17 LTS** 作为开发语言，受益于其长期支持特性与增强的语言特性（如密封类、模式匹配），确保代码的类型安全与可维护性。构建工具选用 **Maven 3.x**，通过标准化的 POM 配置实现依赖管理自动化，结合中央仓库的组件生态，支持传递性依赖解析与版本冲突解决。运行环境基于 **JRE 17+**，利用 Java 平台的“一次编写，到处运行”特性，保障系统在 Windows、Linux 等主流操作系统的兼容性。开发 IDE 支持 **IntelliJ IDEA**，借助其内置的 JavaFX 插件与 Maven 集成功能，实现从代码编写、调试到打包的全流程支持。核心框架选型包括：

- **JavaFX 17：**作为现代化 GUI 框架，其基于 FXML 的声明式界面设计分离了视图定义与逻辑代码，符合“关注点分离”原则。通过 Scene Graph 渲染引擎实现高效的图形渲染，支持硬件加速与 CSS 样式定制（如 `:hover` 伪类、自定义阴影效果），满足应用对界面美观性与交互流畅性的需求。
- **Apache POI 5.2.3：**作为 Office 文档操作的行业标准库，其底层基于 OOXML 格式解析，通过 XWPF 系列类实现对 .docx 文档的精准控制（如段落间距精确到 0.01pt、表格边框线型配置）。针对大文档处理，采用流式 API（`XWPFStreamer`）降低内存占用，支持百万级数据量的文档生成。

1.2.2 关键技术实现



系统在核心功能实现中深度整合 Java 平台的高级特性，解决复杂业务场景下的性能与可靠性挑战。

- 文档处理技术：

基于 Apache POI 的底层 API 构建文档生成引擎，通过 `XWPFDocument` 对象封装文档结构，利用 `XWPFStyles` 实现样式的继承与迁移。**样式复制机制**通过解析源文档的 `CTStyle` 对象树，复制段落样式（`CTParagraphStyle`）、字符样式（`CTCharStyle`）到目标文档，确保模板样式的完整继承；**表格格式化**则通过操作 `CTTblPr`（表格属性）与 `CTTblGrid`（网格结构），实现单元格合并（`gridSpan`）、边框线型（`STBorder`）、行高列宽（`tblW/tblH`）的程序化控制。

- 多线程并发处理：

采用 **ExecutorService** 线程池管理并发任务，核心线程数配置为

`Runtime.getRuntime().availableProcessors() * 2`（默认 16 线程），通过 `ThreadPoolExecutor` 的拒绝策略（`CallerRunsPolicy`）处理过载任务。**异步任务编排**基于 `CompletableFuture` 实现响应式编程，例如文档生成任务分解为“数据加载 - 模板解析 - 内容填充 - 样式应用”四个阶段，通过 `thenCompose()` 方法实现阶段串行执行，结合 `whenCompleteAsync()` 进行结果汇总与 UI 更新。线程安全控制通过 `ReentrantReadWriteLock` 实现数据结构的读写分离，确保多线程环境下 `TreeNode` 对象的并发访问安全。

- 界面技术框架：

- 组件体系：

- **TreeView 控件**：基于 JavaFX 的 `TreeItem` 模型构建项目结构树，通过自定义单元格渲染器（`TreeCell`）实现节点图标（如文件夹 / 文件图标）与状态标记（如未保存星标）的可视化。节

点展开 / 折叠事件通过 `TreeItem.expandedProperty()` 监听，结合 `WeakListener` 避免内存泄漏。

- **TableView 控件**：采用 `ObservableList` 作为数据载体，通过 `TableColumn.setCellValueFactory()` 实现数据绑定，支持列排序（`TableColumn.setSortable(true)`）与过滤（通过 `Predicate` 动态筛选行）。单元格编辑功能通过 `CellFactory` 创建可编辑控件（如 `TextFieldTableCell`），结合 `CommitValueEvent` 实现编辑值的验证与持久化。
- **进度展示**：`ProgressBar` 与 `ProgressIndicator` 控件绑定 `ProgressTask` 的 `progressProperty()`，通过 CSS 自定义进度条颜色（如 `-fx-accent: #4CAF50`），支持不确定模式（`progress=-1`）用于未知耗时任务。

- **布局系统**：

主界面采用 **GridPane+AnchorPane+VBox** 的复合布局模式：左侧项目树通过 `VBox` 容器实现垂直排列，设置最小宽度 250px 与最大宽度 300px 以适应不同屏幕尺寸；右侧内容区域使用 `AnchorPane` 实现控件的四边锚定，确保窗口缩放时控件间距保持一致；整体包裹在 `ScrollPane` 中，支持大内容区域的滚动浏览。响应式设计通过 `Priority` 属性实现控件尺寸弹性分配，例如表格控件的 `Hgrow` 设置为 `ALWAYS`，使其在水平方向填充可用空间。

- **样式定制**：

界面样式基于 CSS3 语法定义，通过 `ExternalStyleSheet` 动态加载样式表。**暗黑主题**通过设置根容器的 `-fx-base: #333333` 与 `-fx-control-inner-background: #444444` 实现整体色调调整；`TreeCell` 控件通过 `-fx-text-fill: white` 提升可读性，结合 `:hover` 伪类改变背景色（`-fx-background-color: #555555`）增强交互反馈。

- **序列化机制**：

采用 **JSON 格式** 实现项目数据持久化，通过 Jackson 库的 `ObjectMapper` 完成 `TreeNode` 对象树的序列化与反序列化。自定义序列化器（`JsonSerializer`）处理树形结构的循环引用问题，通过节点 ID 映射表（`Map<Long, TreeNode>`）避免递归序列化导致的栈溢出。

- **文件存储结构**：

- **项目文件 (.record)**：采用二进制格式（基于 JSON）存储项目元数据、树形结构与业务配置，支持快速加载与增量更新。
- **模板文件 (.docx)**：遵循 Apache POI 的模板规范，通过 `${variable}` 占位符标记数据注入点，支持段落、表格、文本框等多类型模板元素。
- **函数规格文件 (-vmod.docx)**：基于 Apache POI 执行信息表格定位与解析。
- **流程图文件 (-vcht.docx)**：利用 Apache POI 底层对 XML 的支持体系构建自定义解析引擎。

- **日志级别管理**：

采用五级日志分级体系（TRACE/DEBUG/INFO/WARN/ERROR），通过 `logback.xml` 配置文件动态调整输出级别。生产环境默认记录 INFO 级别以上日志，开发环境开启 DEBUG 级别以支持详细调试。日志输出包括控制台（`ConsoleAppender`）与文件（`RollingFileAppender`）双渠道，文件日志按天滚动（`TimeBasedRollingPolicy`），保留最近 7 天的历史记录。

- **日志实现细节**：

基于 SLF4J 日志门面实现框架无关性，具体日志实现选用 Logback-classic。日志记录采用参数化格式（如 `logger.info("文档生成完成, 耗时 {} ms", duration)`），避免字符串拼接带来的性能损耗。异步日志记录通过 `AsyncAppender` 实现，将日志写入操作提交至后台线程，减少业务线程的 I/O 阻塞。

- 异常处理机制：

采用集中式异常处理模式，通过自定义 `GlobalExceptionHandler` 捕获所有未处理异常，结合 AOP 技术实现对业务方法的环绕通知。异常分类遵循“抽象到具体”原则，基类 `SystemException` 封装通用错误码（如 `ERROR_CODE_001` 表示系统内部错误），子类 `DataValidationException`、`DocumentGenerateException` 等扩展特定业务场景的错误信息。资源释放遵循“Try-With-Resources”模式，确保文件流、数据库连接等资源的自动关闭，避免内存泄漏。

1.3 功能设计

各个功能模块的设计都遵循了高内聚低耦合的原则，通过清晰的接口定义实现模块间的协作。设计思想体现了系统的整体架构考虑，而详细设计则展示了具体的实现方案和关键代码结构。

1.3.1 文档生成功能

1.3.1.1 设计思想

文档生成功能整体上采用模板驱动的文档生成方案，通过预定义的模板文件控制文档结构和样式，并在功能实现上采用了模块化处理，将概要设计、详细设计、追踪表等功能解耦。具体的，系统使用游标定位技术精确控制内容插入位置，确保文档生成的准确性。

各个模块之间的生成互相独立，通过并行处理可以提升大量函数详细信息处理效率。

1.3.1.2 详细设计

概要设计：

主要包含“功能需求及设计约束”、“软件模块与单元关系”、“软件模块开发状态”三部分。第一部分的详细内容从模块规格文件（vmod）中的Outline信息提取；第二部分由用户填写的表单信息决定，其中“软件单元序号”即为用户填写的函数名称；第三部分采用默认值填充。

a) 功能需求及设计约束

系统时钟频率设置模块在程序其它功能运行前。

b) 软件模块与单元关系

下表列出本软件模块与下级软件单元之间的包含关系。

表1 系统时钟频率设置模块软件单元列表

序号	软件单元名称	软件单元序号
1	功能 1	func1

c) 软件模块开发状态

本软件模块为新开发，不存在继承问题，基于标准 C 开发，计算机硬件资源为 LCDSP0102。

```
1 public static int overviewDesign(...) {
2     // 1. 需求标题生成
3     XWPFPParagraph newRequestParagraph1 = doc.insertNewParagraph(cursor1);
4     newRequestParagraph1.setStyle(styles.getStyleWithName("2级有标题条").getStyleId());
5
6     // 2. 模块遍历处理
7     for(ModuleNode module : request.getModuleName()) {
8         // 2.1 章节号映射
9         moduleChapterMap1.put(module.getModuleName(),
10             "4.1." + requestIdx + "." + moduleIdx);
11
12         // 2.2 模块标题生成
13         XWPFPParagraph newModuleParagraph = doc.insertNewParagraph(cursor1);
14         newModuleParagraph.setStyle(styles.getStyleWithName("3级有标题条").getStyleId());
15
16         // 2.3 功能需求及约束生成
17         XWPFPParagraph para_a = doc.insertNewParagraph(cursor1);
18         para_a.setIndentationLeft(448);
19         para_a.setIndentationHanging(15);
20     }
21 }
```

追踪表生成：

```
1 public static void trackingTableProcessing(...) {
```

```

2      // 1. 正向追踪表: 需求->模块->函数
3      XWPFTable forwardTrackingTable = tables.get(tables.size() - 3);
4      for(RequestNode request : RequestNodeList) {
5          for(ModuleNode module : request.getModuleName()) {
6              setTableCell(newRow.getCell(2), styles,
7                  moduleChapterMap1.get(module.getModuleName()));
8          }
9      }
10
11     // 2. 逆向追踪表: 模块->需求
12     XWPFTable reverseTrackingTable = tables.get(tables.size() - 2);
13     for(ModuleNode module : request.getModuleName()) {
14         setTableCell(newRow.getCell(0), styles,
15             moduleChapterMap1.get(module.getModuleName()));
16     }
17 }

```

同时提供进度条可视化展示文档生成进度，通过异步处理避免界面卡顿。

```

1  // 进度条更新处理
2  public static void updateProgress(int connFuncNum) {
3      // 计算进度
4      progress += 1.0 / connFuncNum;
5      progressBar.setProgress(progress);
6
7      // UI线程更新显示
8      Platform.runLater(() ->
9          progressNum.setText(String.format("%.2f%%", progress * 100))
10     );
11 }
12
13 // 任务处理框架
14 Task<Void> task = new Task<Void>() {
15     @Override
16     protected Void call() throws Exception {
17         // 执行文档生成
18         OnlyAllDetail.detailDesign_ALL(
19             originalTree,
20             templatePath,
21             generatedDirectoryPath + "\\\" + "生成文档（仅详设）.docx"
22         );
23         return null;
24     }
25 };
26
27 // 进度监控
28 dialog.show();
29 Platform.runLater(() -> {
30     progressBar.setProgress(progress);
31     progressNum.setText("已完成" + progress * 100 + "%");
32     if (isGenerate) {

```

```

33         dialog.close();
34     }
35 });

```

1.3.2 需求管理功能

1.3.2.1 设计思想

需求管理功能采用表格化的需求管理方式，直观展示需求-模块-函数的层级关系，便于用户进行需求的编辑，表中实现了需求条目的动态增删改，支持实时编辑和保存，此外，通过数据绑定确保界面显示与数据模型的同步，同时引入了验证机制防止重复条目和无效输入，系统在处理函数的层级关系时，会递归处理相应的子函数，确保函数间调用关系的正确性。

1.3.2.2 详细设计

设计需求数据结构如下：

```

1  public class RequestNode {
2      private String requestName;           // 需求名称
3      private List<ModuleNode> moduleName; // 关联模块列表
4
5      // 获取当前需求列表
6      public static List<String> getNowRequestList() {
7          List<String> RequestList = new ArrayList<>();
8          for (ModuleNode moduleNode : moduleNodeList) {
9              if (!Objects.equals(moduleNode.getRequestName(), "") &&
10                  !RequestList.contains(moduleNode.getRequestName())) {
11                  RequestList.add(moduleNode.getRequestName());
12              }
13          }
14          return RequestList;
15      }
16  }

```

提供需求编辑功能：

```

1  // 需求列编辑提交事件
2  requestCol.setOnEditCommit(event -> {
3      ModuleNode moduleNode = event.getRowValue();
4      String newRequestName = event.getNewValue();
5
6      // 空值处理
7      if (newRequestName.equals("")) {
8          RequestList = getRequestList();
9          String curRequestName = RequestList.get(RequestList.size() - 1);
10         moduleNode.setRequestName(curRequestName);
11     }
12     moduleNode.setRequestName(newRequestName);
13 });

```

1.3.3 函数关联功能

1.3.3.1 设计思想

函数关联功能采用树形结构展示函数调用关系，直观反映函数间的层级关系，方便用户观察函数间调用关系，并通过颜色标识区分函数的选择状态和详细设计状态。除此之外，还实现了函数节点的展开/折叠，优化大规模函数树的显示，支持函数节点的动态关联和解除关联。

1.3.3.2 详细设计

函数节点数据结构设计：

```
1 public class TreeNode {
2     private String name;
3     private List<TreeNode> children;
4     private int colorNum = 0;    // 节点颜色状态
5     private boolean isExpanded = false; // 展开状态
6
7     // 节点展开处理
8     public TreeNode expandChildren(TreeNode root, TreeNode curNode,
9         TreeNode curroot) {
10         TreeNode foundNode = searchNode(root, curNode);
11         if (foundNode != null && foundNode.getChildren().size() != 0) {
12             curNode.setExpanded(true);
13             for (TreeNode child : foundNode.getChildren()) {
14                 TreeNode curchild = new TreeNode(child.getName());
15                 curNode.addChild(curchild);
16                 expandChildren(root, curchild, curroot);
17             }
18         }
19         updateColor();
20         return curroot;
21     }
22 }
```

颜色状态更新处理：

```
1 public static void updateColor() {
2     resetColor(curTree);
3     selectNodeList = getNowSelectNodeList();
4
5     // 更新选中节点
6     for (String s : selectNodeList) {
7         List<TreeNode> resultList = curTree.searchNodeList(
8             curTree, s, new ArrayList<>());
9         for (TreeNode node : resultList) {
10             node.setColorNum(1);
11         }
12     }
13
14     // 更新详细设计节点
```

```

15     for(String node : detailDesignedNodeList) {
16         List<TreeNode> resultList = curTree.searchNodeList(
17             curTree, node, new ArrayList<>());
18         for(TreeNode result : resultList) {
19             if(!selectNodeList.contains(result.getName())) {
20                 result.setColorNum(2);
21             }
22         }
23     }
24 }

```

1.4 前端设计

1.4.1 界面布局设计

1.4.1.1 设计思想

系统整体界面采用三栏式布局，左侧为文件树，中间为设计区，右侧为调用关系图，采用深色主题设计，从而确保界面元素的视觉层次，界面风格统一，使用响应式设计，确保界面元素自适应调整，并可以通过滚动面板优化大量数据的显示，该实现是基于模块化思想的组件设计，便于维护和复用。

1.4.1.2 详细设计

布局容器管理：

```

1  // 1. 根布局容器
2  public static GridPane rootPane = new GridPane();
3  public static GridPane rootGridPane = new GridPane();
4
5  // 2. 主要区域划分
6  public static Pane treePane = new Pane();           // 树形展示区
7  public static Pane nodePane = new Pane();           // 节点展示区
8  public static ScrollPane scrollPane = new ScrollPane(); // 滚动区域
9  public static GridPane fileUploadGridPane = new GridPane(); // 文件上传区
10
11 // 3. 布局配置
12 fileTreeVBox.setMinWidth(250);
13 fileTreeVBox.setMaxWidth(300);
14 fileTreeVBox.setMinHeight(980);
15 treeView.setMinHeight(980);

```

样式风格设计：

```

1  // 1. 全局样式设置
2  scene.getStylesheets().add(myCssFile);
3
4  // 2. 组件样式
5  tableView.setStyle("-fx-background-color: rgb(66,66,66);");
6  fileTreeVBox.setStyle("-fx-background-color: rgb(66, 66, 66); " +
7      "-fx-border-color: #333333; -fx-border-width: 0 2 0 0;");

```

```

8
9 // 3. 节点状态样式
10 switch (node.getColorNum()) {
11     case 1 -> nodeBox.setFill(Color.rgb(255, 182, 193)); // 选中状态
12     case 2 -> nodeBox.setFill(Color.rgb(152, 251, 152)); // 详细设计状态
13     case 3 -> nodeBox.setFill(Color.rgb(119, 136, 153)); // 未选中状态
14     default -> nodeBox.setFill(Color.rgb(176, 196, 222)); // 默认状态
15 }

```

进度条展示：

```

1 // 1. 进度条组件
2 public static ProgressBar progressBar = new ProgressBar();
3 public static Label progressNum = new Label();
4 public static double progress = 0;
5
6 // 2. 进度更新实现
7 public static void updateProgress(int connFuncNum) {
8     progress += 1.0 / connFuncNum;
9     progressBar.setProgress(progress);
10    Platform.runLater(() ->
11        progressNum.setText(String.format("%.2f%%", progress * 100))
12    );
13 }
14
15 // 3. 进度对话框
16 Dialog<Void> dialog = new Dialog<>();
17 VBox dialogVbox = new VBox(10);
18 dialogVbox.getChildren().addAll(
19     new Label("正在生成文档..."),
20     progressBar,
21     progressNum
22 );
23 dialog.getDialogPane().setContent(dialogVbox);

```

1.4.2 交互控件设计

1.4.2.1 设计思想

系统实现可编辑的表格控件，支持实时数据更新，通过列工厂模式定制单元格的展示和编辑行为，同时支持数据验证和错误提示，实现表格数据与模型的双向绑定。

在函数展示控件中，采用自定义渲染的树形结构展示函数调用关系，实现节点的展开/折叠功能，通过颜色编码显示节点状态，并且支持节点间连线的动态绘制，提高系统整体性能。

1.4.2.2 详细设计

表格交互控件设计：

```

1 // 1. 表格基础配置
2 TableView<ModuleNode> tableView = new TableView<>(data);

```

```

3 tableView.setSortPolicy(table -> false);
4 tableView.setEditable(true);
5 tableView.setStyle("-fx-background-color: rgb(66, 66, 66);");
6
7 // 2. 需求列配置
8 TableColumn<ModuleNode, String> requestCol = new TableColumn<>("需求");
9 requestCol.setPrefWidth(125);
10 requestCol.setEditable(true);
11 requestCol.setCellValueFactory(new PropertyValueFactory<>("requestName"));
12
13 // 3. 编辑提交处理
14 requestCol.setOnEditCommit(event -> {
15     ModuleNode moduleNode = event.getRowValue();
16     String newRequestName = event.getNewValue();
17     if (newRequestName.equals("")) {
18         RequestList = getRequestList();
19         String curRequestName = RequestList.get(RequestList.size() - 1);
20         moduleNode.setRequestName(curRequestName);
21     }
22     moduleNode.setRequestName(newRequestName);
23 });

```

树形组件：

```

1 // 1. 节点渲染参数
2 private static final double BOX_HEIGHT = 18; // 节点高度
3 private static final double HORIZONTAL_GAP = 18; // 水平间距
4 private static final double VERTICAL_GAP = 2; // 垂直间距
5 private static final double TEXT_PADDING = 2; // 文本内边距
6
7 // 2. 节点绘制实现
8 public static double drawNodeHierarchy(Pane pane, TreeNode tree,
9     TreeNode node, double x, double y) {
10
11     // 2.1 计算节点尺寸
12     Text nodeName = new Text(node.getName());
13     double boxWidth = nodeName.getLayoutBounds().getWidth() + 2 * TEXT_PADDING;
14     double maxY = y + BOX_HEIGHT;
15
16     // 2.2 创建节点容器
17     Rectangle nodeBox = new Rectangle(x, y, boxWidth, BOX_HEIGHT);
18     nodeBox.setArcWidth(5);
19     nodeBox.setArcHeight(5);
20
21     // 2.3 设置节点颜色
22     switch (node.getColorNum()) {
23         case 1 -> nodeBox.setFill(Color.rgb(255, 182, 193));
24         case 2 -> nodeBox.setFill(Color.rgb(152, 251, 152));
25         case 3 -> nodeBox.setFill(Color.rgb(119, 136, 153));
26         default -> nodeBox.setFill(Color.rgb(176, 196, 222));
27     }

```

```

28
29 // 2.4 添加展开/折叠控件
30 if (!foundNode.getChildren().isEmpty() && !node.getExpandedState()) {
31     Circle plusSignCircle = new Circle(plusSignX, plusSignY, 5);
32     plusSignCircle.setStyle("-fx-font-size: 6;-fx-font-weight: bold;-fx-fill:
#333333");
33     pane.getChildren().add(plusSignCircle);
34 }
35 }

```

按钮控件：

```

1 // 1. 按钮控件配置
2 Button addButton = new Button("添加概设");
3 ImageView addButtonIcon = new ImageView(new Image(
4     demoApplication.class.getResourceAsStream("/image/icon/add.png")));
5 addButtonIcon.setFitWidth(16);
6 addButtonIcon.setFitHeight(16);
7 addButton.setGraphic(addButtonIcon);
8
9 // 2. 按钮事件处理
10 addButton.setOnAction(event -> {
11     ModuleNode moduleNode = new ModuleNode("");
12     if (selectedModule != null) {
13         int selectedIndex = data.indexOf(selectedModule);
14         moduleNode.setRequestName(selectedModule.getRequestName());
15         data.add(selectedIndex + 1, moduleNode);
16     } else {
17         String curRequestName = getLastValidRequestName(requestList);
18         ModuleNode newNode = new ModuleNode(curRequestName);
19         data.add(newNode);
20     }
21     tableView.refresh();
22 });
23
24 // 3. 对话框设计
25 Dialog<Pair<String, String>> dialog = new Dialog<>();
26 dialog.setTitle("添加函数");
27 dialog.setHeaderText(null);
28
29 // 3.1 对话框内容布局
30 GridPane grid = new GridPane();
31 grid.setHgap(10);
32 grid.setVgap(10);
33 grid.setPadding(new Insets(20, 150, 10, 10));
34
35 // 3.2 输入控件
36 TextField englishName = new TextField();
37 TextField chineseName = new TextField("");
38 grid.add(new Label("英文名:"), 0, 0);
39 grid.add(englishName, 1, 0);

```



```
40 grid.add(new Label("中文名:"), 0, 1);
41 grid.add(chineseName, 1, 1);
```

#